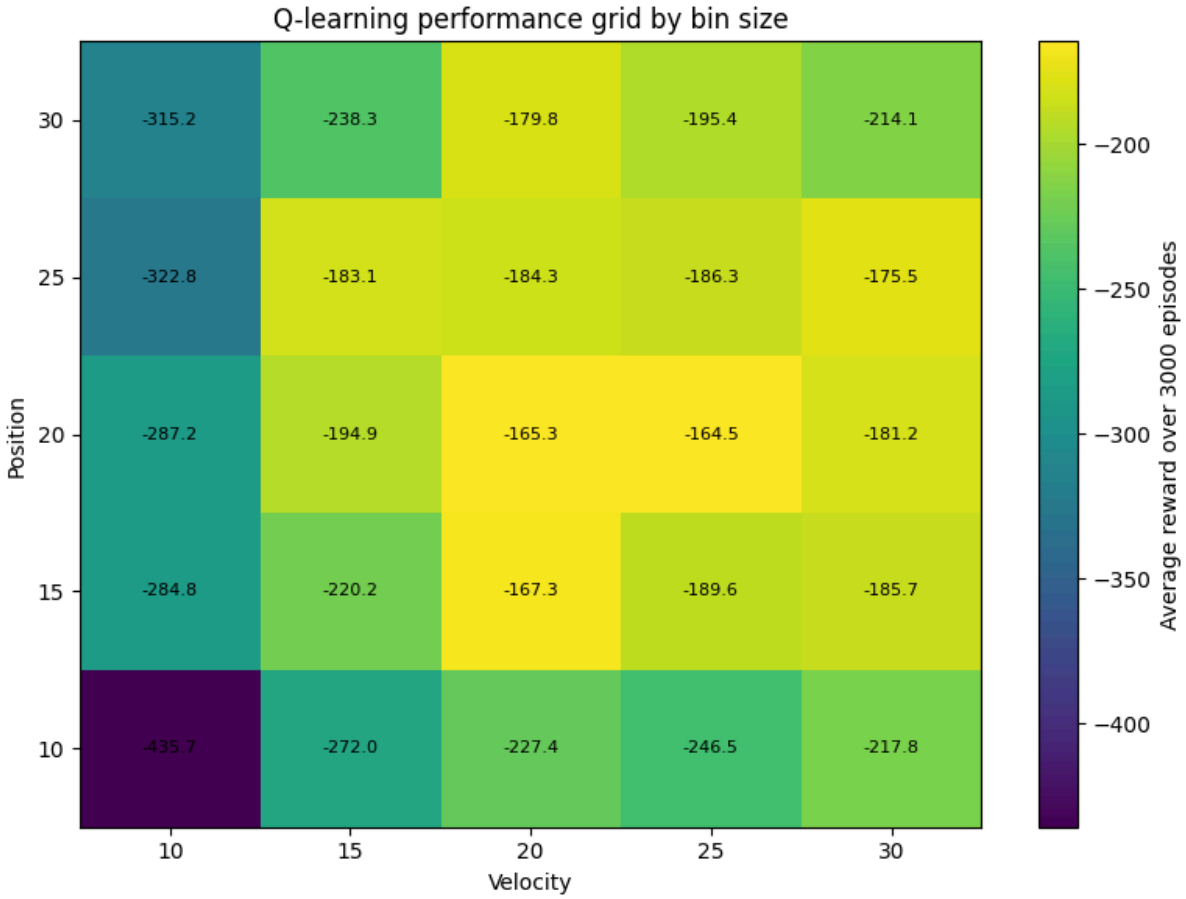
Problem set 4 report

Exercise 4 - Feature engineering

The first step of this exercise was transforming the data to be better suited for Q-learning. The continuous states had to be converted into a discrete form. We created a state discretization function that normalizes a state to a value between 0 and 1, which can then be scaled to the number of bins chosen (discrete states).

Then a q-learning function was created, and parameters for learning rate, discount factor and exploration rate were set. In this case learning rate was 0.1, discount factor was 0.95 and the exploration rate used epsilon decay (starting at 1 and decreasing for each each episode).

To train and compare, a bin range was made [10, 15, 20, 25, 30]. Each possible bin touple (position, velocity) was trained with 3000 episodes. In the figure below the results can be seen, displayed in a grid format.

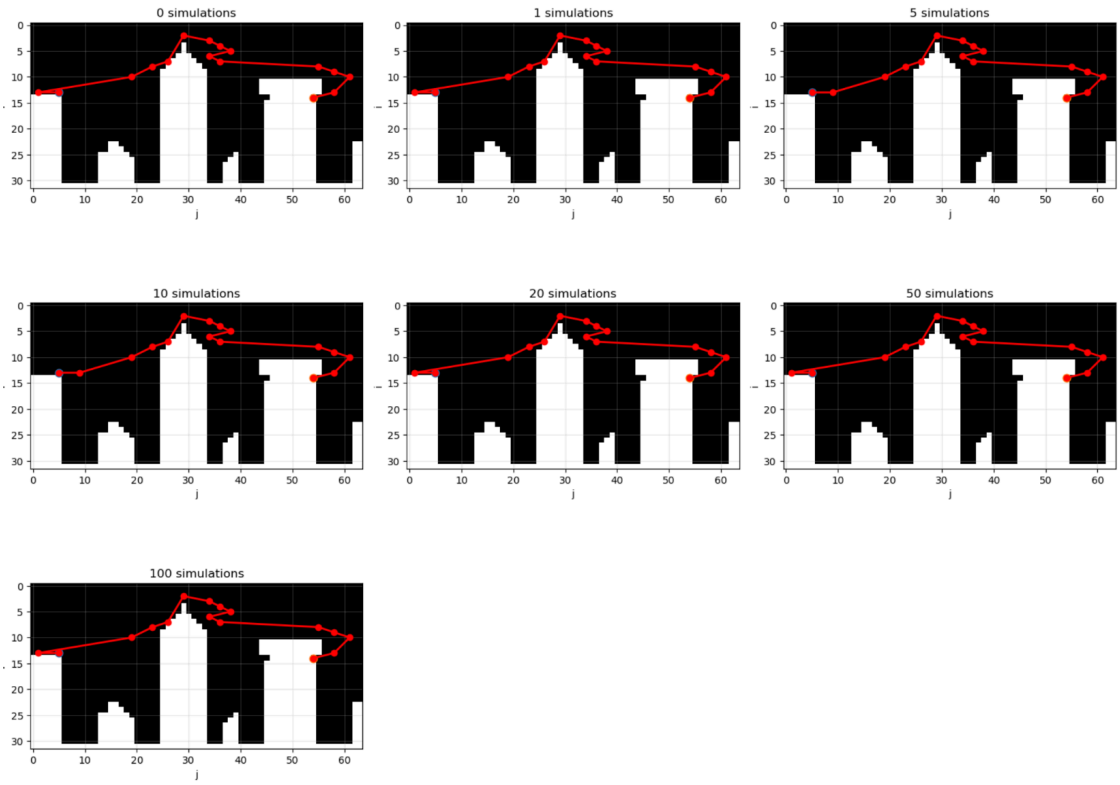


The most promising touples were chosen and trained with 10000 episodes. Out of those the best result was achieved with (20,20), with an average reward being -154.

There are some feature engineering approaches that could improve performance. One option would be non-uniform discretization. This would provide more bins to more impactful regions of the state space.

Exercise 5 - Planning

To evaluate the effect of planning in the Dyna architecture we varied the number of simulated updates performed after each real experience update. More specifically we tested planning steps of 0,1,5,10,20,50 and 100. For each setting we trained the agent and extracted the greedy policy, then we simulated the resulting trajectory starting from the initial state.

The results show that the learned trajectories are almost identical across all planning settings. In each case the drone successfully reaches the landing zone while avoiding obstacles and exploiting the wind currents. The performance metrics are also nearly the same, with a final reward of 1.0 and an average reward of 1.0.This occurs because the dataset already contains successful transitions that lead to the goal. Even without planning, the Q-learning updates from the real experience are sufficient to propagate the reward information through the relevant states. Increasing the number of simulated updates therefore does not significantly change the final learned policy.

Under are the different trajectories, the only difference seems to be that in some of the cases the first step is 3 (right) instead of -3 (left). In this environment the available data already captures the optimal trajectory well, making the learned policy largely insensitive to the number of planning simulations.